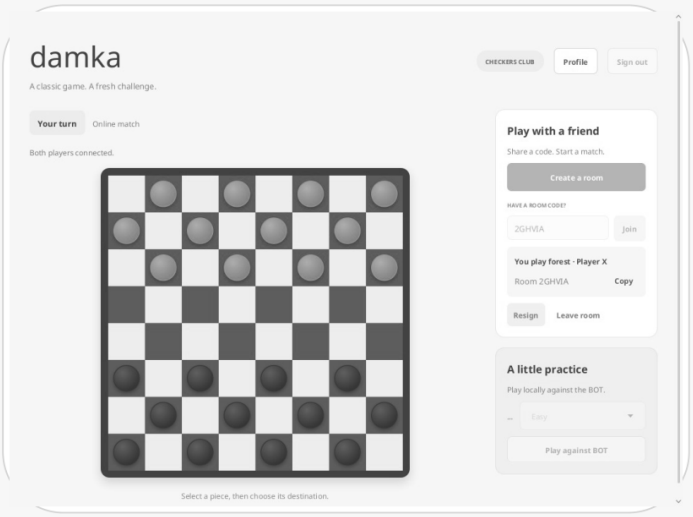
Damka

Checkers, connected.

TCP multiplayer · Local BOT
Accounts, statistics and history



Muthanna Farhat
Rabea Abu Aad
Supervisor: Inbal Ronen

Slide 1

One game, two ways to play

Keep both screens in sync — and remember the result.

Online matches

Two computers
One room code

Local BOT

Three difficulty levels
Offline practice

Personal profiles

Statistics
Match history

One shared rules engine: Game

Slide 2

Built with Java — from interface to storage

Java 17

Object-oriented game and server code

JavaFX + CSS

Desktop screens, styling and animation

TCP / java.net

Socket / ServerSocket · UTF-8

SQLite database

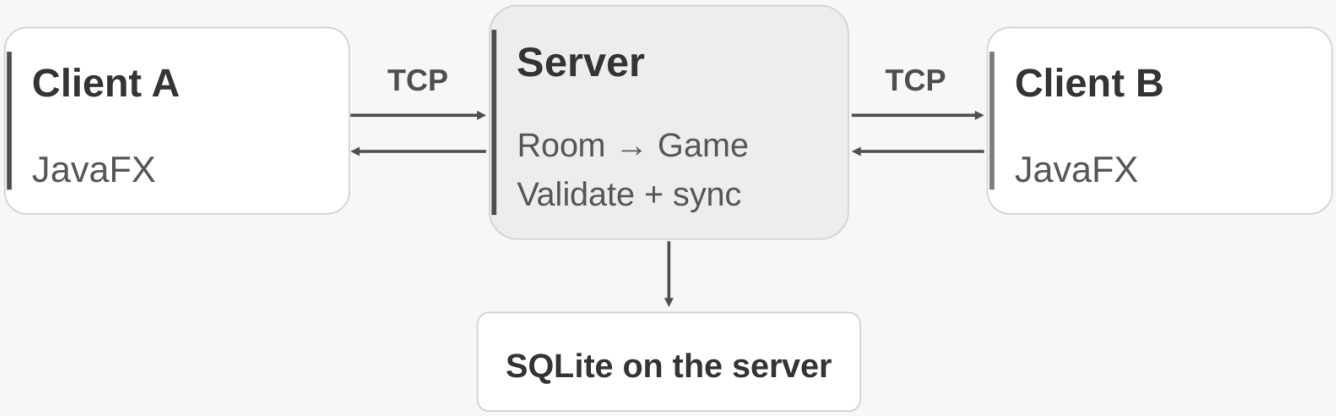
Accounts, results and history

JDBC lets the server’s Java code read and write SQLite.

Build: Gradle 8.8 **Tests: JUnit 5** **Version control: Git + GitHub**

Slide 3

Why a client–server design?

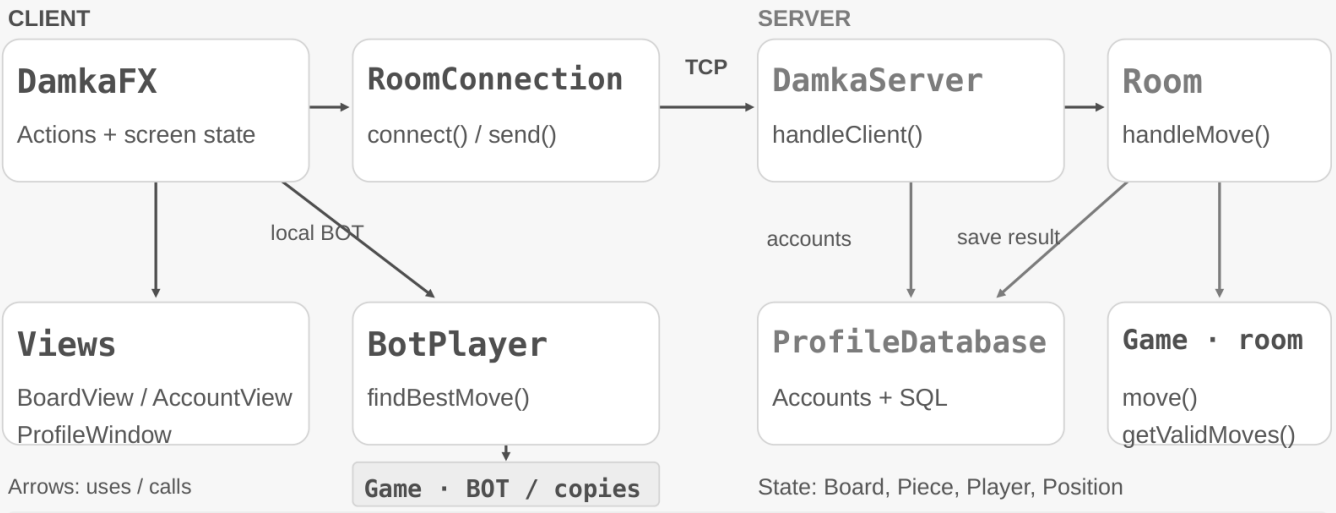


Choice: persistent TCP for two-way updates

Alternative: HTTP polling

Slide 4

Main classes and their responsibilities



Game rules are reused; rooms, local BOT and search use separate game instances.

Slide 5

From a click to both screens

Client A

Select a piece and destination

```
MOVE 5 0 4 1
```

Server

Room: identity + turn

Game: check move rules

```
game.move(from, to)
```

Both clients

Accepted move

```
1  MOVE_APPLIED
2  BOARD
3  TURN
```

Animate → display the accepted state

UTF-8 · one message per line · network work off the UI thread

Slide 6

Minimax: how the BOT chooses

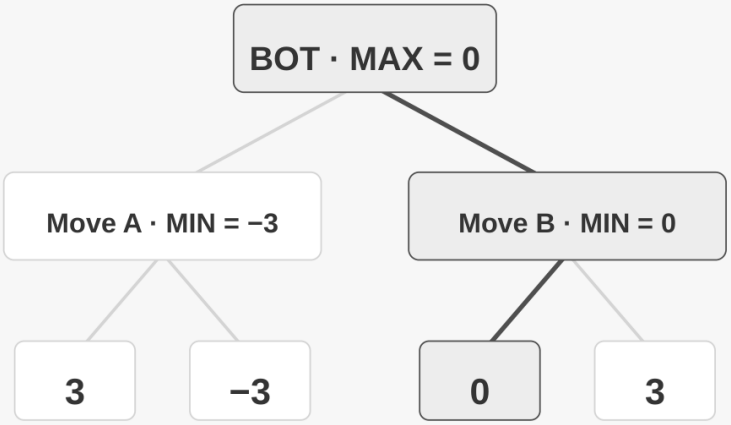
Shared game rules

Legal moves on copies

Piece = 3 King = 6

Depth: 1 / 2 / 3

Alpha–Beta reduces unnecessary search.



Illustrative search · 0 is a board score, not a match result.

Final choice: move B — MAX(−3, 0) = 0

Slide 7

Accounts that remember results

Player profile

Illustrative account

ONLINE	BOT
1 win / 1 match	0 matches

Match history

Online	WIN	Resignation

`profiles`

Username · salt · hash

`game_results`

Players · result · mode · time

Saved results → statistics

Slide 8

How we checked the system

48 automated tests

Game rules

Invalid moves, turns and capture chains

TCP behavior

Two real connections
Matching board updates

Stored results

Authentication
Persistence + deduplication

Manual UI checks: two windows, error messages and movement animation

Automated count: documented passing run, 12 Oct 2026

Slide 9

Challenges and what we learned

Learning checkers rules

Movement, mandatory captures, capture chains and promotion.

Learning JavaFX

Learn a new UI library and connect buttons and board clicks to actions.

TCP in practice

Apply course theory to real client–server connections and messages.

From HTML to JavaFX

Adapt familiar web layouts to JavaFX components and user events.

Slide 10

Responsibilities and AI support

Rabea Abu Aad

Game logic

Movement and captures,
turns and promotion,
game ending and winner.

Muthanna Farhat

TCP communication

Client–server connections,
requests and replies,
synchronized game state.

Extensive AI support: Codex + GitHub Copilot

Learning, planning, writing and revising code, UI, tests and documentation.

Define requirements → review suggestions → run and test → adjust

Slide 11

What we built and learned

One authoritative game.
Two synchronized clients.
Persistent player results.

Next: chat

Inside the room
Over existing TCP

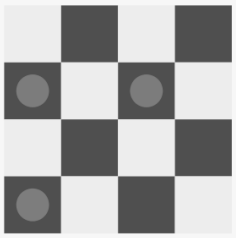
Learning: JavaFX, practical networking,
and integration across UI, rules and storage.

github.com/MuthannaFrhat/damka-game

Slide 12

THANK YOU

Questions & discussion



github.com/MuthannaFrhat/damka-game

Slide 13